Academic Project Report

Design and Simulation of a 5-Stage Pipelined RISC-V Processor

---

PROJECT REPORT



Submitted by

Sanjeev

Department	Electronics and Communication Engineering
Academic Year	2026–2027
Course / Subject	_____
Project Guide	_____

Verilog HDL Icarus Verilog GTKWave

2026

Certificate

This is to certify that the project report entitled "Design and Simulation of a 5-Stage Pipelined RISC-V Processor" presents the design, implementation, organization, simulation, and verification work carried out using Verilog Hardware Description Language.

The project focuses on developing an educational processor architecture based on a selected subset of the RISC-V instruction set. The implementation is organized into five pipeline stages: Instruction Fetch, Instruction Decode, Execute, Memory Access, and Writeback. It includes pipeline registers, forwarding logic, hazard detection, branch and jump handling, memory operations, and register writeback.

The design was simulated using Icarus Verilog. Individual modules and processor-level behavior were evaluated through dedicated testbenches. The report records the implementation approach and the observed results of the integration tests.

Project Student: Sanjeev

Department: Electronics and Communication Engineering

Project Guide: _____

Date: _____ **Signature:** _____

Declaration

I declare that this report documents the design and simulation work undertaken for the project entitled “Design and Simulation of a 5-Stage Pipelined RISC-V Processor”.

The processor was developed using Verilog HDL with a modular design approach. The RTL modules were organized separately and integrated into a top-level processor module. The verification process involved individual testbenches and processor-level integration tests covering arithmetic, memory operations, data dependencies, branches, and jumps.

The implementation is an educational processor supporting a selected subset of RISC-V functionality. It is not claimed to be a fully compliant implementation of the complete RV32I specification.

Name: Sanjeev

Signature: _____

Date: _____

Acknowledgement

I would like to express my gratitude to everyone who contributed to the completion of this project.

The development of a pipelined processor using Verilog HDL provided an opportunity to understand the relationship between computer architecture and digital hardware implementation. The project involved studying instruction formats, datapath organization, pipeline registers, control signals, register operations, arithmetic logic, memory organization, data forwarding, and hazard handling.

I also acknowledge the importance of simulation and verification tools such as Icarus Verilog and GTKWave in understanding and evaluating hardware designs. The process of debugging integration tests helped develop a practical understanding of timing relationships and dependencies between pipeline stages.

Abstract

This project presents the design and simulation of an educational 32-bit RISC-V processor organized as a five-stage pipeline using Verilog HDL. The objective is to implement a modular processor architecture that demonstrates instruction processing through Instruction Fetch, Instruction Decode, Execute, Memory Access, and Writeback stages.

The implementation consists of separate RTL modules for the arithmetic logic unit, register file, immediate generator, control unit, instruction and data memories, pipeline registers, forwarding logic, hazard detection, and stage-level datapath operations. The supported instruction subset includes ADD, SUB, AND, OR, XOR, ADDI, LW, SW, BEQ, BNE, and JAL.

The pipeline includes forwarding paths from later stages, load-use hazard detection, stalling and bubble insertion, branch and jump redirection, and flushing of instructions from a wrong execution path. A same-cycle register-file write/read bypass is included to make a newly written value visible to a matching read.

Simulation was performed using Icarus Verilog, while GTKWave was used to inspect generated VCD waveforms. Three processor-level integration tests were recorded as passing. The main program produced $x1 = 10$, $x2 = 20$, $x3 = 30$, and $x4 = 30$. The branch and jump test verified the expected register values and skipped instructions. The load-use test produced $x1 = 42$, $x2 = 43$, and $x3 = 85$.

The project provides a practical foundation for understanding pipelined datapaths, hazard management, RTL development, and simulation-based hardware verification.

Keywords: RISC-V, Verilog HDL, five-stage pipeline, processor datapath, forwarding, load-use hazard, branch handling, Icarus Verilog, GTKWave.

Contents

Certificate **i**

Declaration **ii**

Acknowledgement **iii**

Abstract **iv**

1 Introduction **1**

1.1 Background 1

1.2 Project Motivation 1

1.3 Project Objectives 1

1.4 Project Scope 2

2 RISC-V Architecture Fundamentals **3**

2.1 Introduction to RISC-V 3

2.2 General-Purpose Registers . . . 3
2.3 Instruction Formats . . . 3
2.4 Implemented Instruction Subset . . . 4
3 Overall Processor Architecture 5
3.1 Five-Stage Pipeline Organization . . . 5
3.2 Instruction Execution Flow . . . 5
3.3 Pipeline Registers . . . 5
3.4 Pipeline Overlap . . . 5
4 Program Counter and Instruction Fetch 6
4.1 Program Counter . . . 6
4.2 Instruction Memory . . . 6
4.3 Next-PC Selection . . . 6
4.4 Fetch-Stage Integration . . . 6
5 Instruction Decode and Control Unit 7
5.1 Instruction Decoder . . . 7
5.2 Immediate Generator . . . 7
5.3 Control Unit . . . 7
5.4 ID/EX Pipeline Register . . . 7
6 Register File and Execute Stage 8
6.1 Register File Overview . . . 8
6.2 Read and Write Operations . . . 8
6.3 Same-Cycle Write/Read Bypass . . . 8
6.4 Arithmetic Logic Unit . . . 8
6.5 ALU Operand Selection . . . 9
6.6 Branch Comparison . . . 9
7 Memory Access and Writeback 10
7.1 Data Memory Organization . . . 10
7.2 Load Operation . . . 10
7.3 Store Operation . . . 10
7.4 Writeback Selection . . . 10
7.5 Memory Scope . . . 10
8 Data Hazards and Forwarding 11
8.1 Data Hazards . . . 11
8.2 Forwarding . . . 11
8.3 Forwarding Sources . . . 11
8.4 Load-Use Hazard . . . 11
8.5 Stalling and Bubble Insertion . . . 11
9 Branch, Jump and Pipeline Flushing 12
9.1 Conditional Branches . . . 12
9.2 JAL Instruction . . . 12
9.3 Redirect Logic . . . 12
9.4 Pipeline Flushing . . . 12

9.5 Control-Hazard Verification 12
10 RTL Module Organization and Integration **13**
10.1 RTL Organization 13
10.2 Top-Level Integration 13
10.3 Project Directory Structure 13
11 Verification and Testing **15**
11.1 Verification Strategy 15
11.2 Module-Level Testing 15
11.3 Processor-Level Testbenches 15
11.4 Simulation Environment 15
12 Main Processor Integration Results **16**
12.1 Test Program 16
12.2 Observed Results 16
12.3 Result Interpretation 16
13 Branch, Jump and Load-Use Test Results **17**
13.1 Branch and Jump Test 17
13.2 Load-Use Hazard Test 17
14 Waveform Analysis, Limitations and Conclusion **19**
14.1 Waveform Analysis 19
14.2 Limitations 19
14.3 Future Improvements 19
14.4 Conclusion 19
References **21**

List of Figures

3.1 Five-stage pipeline and inter-stage registers 5

List of Tables

2.1 General-purpose register overview 3
2.2 Major instruction formats 4

5.1 Important control signals 7

6.1 ALU operation summary 8

10.1 RTL module groups 13

12.1 Main processor test results 16

13.1 Branch and jump integration results 17
13.2 Load-use integration results 17

Chapter 1

Introduction

1.1 Background

A processor is a digital system responsible for executing instructions and controlling the operations of a computing system. Its basic functions include fetching instructions, interpreting instruction fields, reading operands, performing computations, accessing memory, and updating architectural state.

A simple processor may complete one instruction before beginning the next. A pipelined processor divides instruction processing into stages so that several instructions can be active at different stages during the same clock cycle. This overlap can improve instruction throughput, but it introduces data and control dependencies that must be handled by additional hardware.

The RISC-V architecture provides an open instruction set architecture suitable for academic study and hardware experimentation. This project implements an educational 32-bit processor based on a selected RISC-V instruction subset.

1.2 Project Motivation

The project was undertaken to understand how instruction-set operations are translated into hardware behavior. Rather than treating a processor as a black box, the design is divided into independently understandable modules and pipeline stages.

The five-stage organization makes it possible to study how instructions overlap, how intermediate values are stored, and how dependencies are resolved. It also provides practical experience with Verilog RTL, simulation, waveform inspection, and debugging.

1.3 Project Objectives

The main objectives are:

1. Understand the fundamentals of the RISC-V instruction architecture.
2. Design a 32-bit processor datapath organized into five stages.
3. Implement the processor components using modular Verilog HDL.
4. Connect the stages using pipeline registers.
5. Implement forwarding and load-use hazard detection.
6. Handle branch and jump redirection using pipeline flushing.
7. Develop testbenches for individual modules and integrated operation.
8. Verify the design through simulation and waveform inspection.

1.4 Project Scope

The project focuses on an educational five-stage processor supporting selected arithmetic, logical, immediate, memory, branch, and jump instructions. It does not attempt to implement every instruction or privileged feature in the complete RISC-V specification. The reported test results represent directed simulation evidence rather than formal verification or complete ISA compliance.

Chapter 2

RISC-V Architecture Fundamentals

2.1 Introduction to RISC-V

RISC-V is an open instruction set architecture based on Reduced Instruction Set Computer principles. It defines the interface between software and processor hardware. The RV32 family uses 32-bit general-purpose registers and a 32-bit integer execution model.

The processor in this project uses 32-bit instruction words and a 32-bit datapath. Instructions identify operations, source registers, destination registers, and immediate fields depending on the instruction format.

2.2 General-Purpose Registers

The architecture defines 32 general-purpose registers, each 32 bits wide. Register $x0$ is hardwired to zero. The remaining registers are available for operands, addresses, and results.

Table 2.1: General-purpose register overview

Register	ABI name	Typical purpose
$x0$	zero	Constant zero
$x1$	ra	Return address
$x2$	sp	Stack pointer
$x3$	gp	Global pointer
$x4$	tp	Thread pointer
$x5 - x7$	t0-t2	Temporary registers
$x8 - x9$	s0-s1	Saved registers
$x10 - x17$	a0-a7	Arguments and return values
$x18 - x27$	s2-s11	Saved registers
$x28 - x31$	t3-t6	Temporary registers

2.3 Instruction Formats

RISC-V uses several instruction formats. Register-register arithmetic uses the R-type format. Immediate arithmetic and loads use I-type fields. Stores use S-type fields, conditional branches use B-type fields, and JAL uses the J-type format.

Table 2.2: Major instruction formats

Format	Typical use	Role in this project
R-type	Register operations	ADD, SUB, AND, OR, XOR
I-type	Immediate and loads	ADDI and LW
S-type	Stores	SW
B-type	Conditional branches	BEQ and BNE
J-type	Jump	JAL

2.4 Implemented Instruction Subset

The supported instruction functionality includes ADD, SUB, AND, OR, XOR, ADDI, LW, SW, BEQ, BNE, and JAL. Other instructions are outside the scope of the current implementation.

Chapter 3

Overall Processor Architecture

3.1 Five-Stage Pipeline Organization

The processor is organized into five stages. Each stage performs a portion of instruction processing, and pipeline registers preserve intermediate information between adjacent stages.

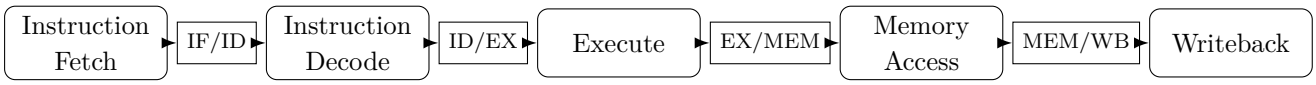


Figure 3.1: Five-stage pipeline and inter-stage registers

3.2 Instruction Execution Flow

The general instruction flow is:

1. Fetch the instruction using the program counter.
2. Decode its fields and read source operands.
3. Execute the selected ALU or branch operation.
4. Access data memory for load or store instructions.
5. Select the result and update the destination register.

3.3 Pipeline Registers

Four pipeline registers separate the five stages:

- IF/ID stores the fetched instruction and associated PC information.
- ID/EX stores operands, immediate values, register indices, and control signals.
- EX/MEM stores the ALU result, store data, destination register, and memory controls.
- MEM/WB stores the information required for final writeback.

3.4 Pipeline Overlap

During normal operation, different instructions occupy different stages at the same time. For example, one instruction can be in Execute while a later instruction is in Decode and another is being fetched. This overlap is the main advantage of pipelining. Dependencies and control-flow changes require stalls, forwarding, and flushing to maintain correct behavior.

Chapter 4

Program Counter and Instruction Fetch

4.1 Program Counter

The program counter stores the address of the instruction being fetched. For sequential execution, the next address is normally:

$$PC_{\text{next}} = PC + 4$$

The PC is updated according to the clock and the control signals associated with reset, stall, and redirection.

4.2 Instruction Memory

Instruction memory supplies the 32-bit instruction associated with the fetch address. The testbench can initialize instruction locations to create directed programs for processor-level verification. Unused locations are initialized with NOP instructions in the documented program setup.

4.3 Next-PC Selection

The processor normally advances to the next sequential address. When a branch is taken or a jump is executed, the target address is selected instead. The top-level redirect condition is:

$$redirect = ex_jump \vee ex_branch_taken$$

The jump target is selected when the jump control is asserted; otherwise the branch target is selected for a taken branch.

4.4 Fetch-Stage Integration

The fetch-stage module internally instantiates the program counter, instruction memory, and IF/ID register. The top-level processor connects to the fetch-stage interface and does not add a duplicate IF/ID register.

Chapter 5

Instruction Decode and Control Unit

5.1 Instruction Decoder

The decoder extracts fields from the 32-bit instruction. Important fields include the opcode, destination register, source register indices, and function fields. These values are used by the control unit, register file, and immediate generator.

5.2 Immediate Generator

Immediate values are encoded in different instruction bit positions. The immediate generator reconstructs the value required by the instruction and sign-extends it where appropriate. The generated immediate is used for arithmetic, effective-address calculation, branch displacement, and jump displacement.

5.3 Control Unit

The control unit interprets the instruction opcode and relevant function fields and produces datapath control signals. These include ALU operation selection, ALU source selection, memory read/write enables, register write enable, branch control, jump control, and writeback selection.

Table 5.1: Important control signals

Signal	Function
reg_write	Enables destination register writeback
mem_read	Indicates a memory read operation
mem_write	Indicates a memory write operation
mem_to_reg	Selects memory data for writeback
alu_src	Selects the second ALU operand
branch	Indicates a conditional branch
jump	Indicates a jump operation
alu_control	Selects the ALU operation

5.4 ID/EX Pipeline Register

The ID/EX register stores the decoded operands, immediate, register indices, and control signals. It also supports flushing to insert a bubble when a load-use hazard or redirect requires invalidating the current instruction slot.

Chapter 6

Register File and Execute Stage

6.1 Register File Overview

The register file contains 32 registers, each 32 bits wide. It provides two read ports and one write port. The decode stage reads source operands, while the writeback path updates the destination register.

Register $x0$ always reads as zero, and writes to it are suppressed.

6.2 Read and Write Operations

The two read ports select operands using the source register indices:

$$ReadData1 = Register[rs1], \qquad ReadData2 = Register[rs2]$$

A destination register is updated when the write enable is asserted and the destination is not $x0$.

6.3 Same-Cycle Write/Read Bypass

The final register-file implementation includes a bypass condition. If a write is enabled, the destination is nonzero, and a read address matches the write address, the read output selects the current write data. Otherwise, it selects the stored register value. This makes the newest write visible to a matching read.

6.4 Arithmetic Logic Unit

The ALU accepts two 32-bit operands and an operation control signal. It supports the operations required by the implemented subset:

Table 6.1: ALU operation summary

Operation	Description
ADD	Addition
SUB	Subtraction
AND	Bitwise AND
OR	Bitwise OR
XOR	Bitwise XOR

6.5 ALU Operand Selection

The first operand is normally supplied by a source register or a forwarded value. The second operand is selected between register data and an immediate. Forwarding selections can replace stale operand values with newer results from later pipeline stages.

6.6 Branch Comparison

The branch comparator evaluates the source operands for BEQ and BNE. BEQ is taken when the operands are equal, while BNE is taken when they differ.

Chapter 7

Memory Access and Writeback

7.1 Data Memory Organization

The data memory is byte-addressable and follows little-endian byte ordering. The documented integration programs exercise aligned 32-bit word loads and stores. The memory stage receives an effective address and store data from the execute path.

7.2 Load Operation

A load instruction calculates an effective address using a base register and an immediate offset. The memory stage reads the addressed word and carries the result through MEM/WB toward the writeback multiplexer.

7.3 Store Operation

A store instruction calculates its effective address and supplies the source-register value as store data. The memory write enable controls the update. A store does not write an ALU result into the register file.

7.4 Writeback Selection

The writeback multiplexer selects the value to be written into the destination register. The implemented paths include the ALU result, memory data, and $PC + 4$ for JAL. The final writeback data is also used by the MEM/WB forwarding path.

7.5 Memory Scope

The recorded integration tests establish aligned word load/store behavior. They do not establish complete support for byte and halfword accesses or all possible alignment cases.

Chapter 8

Data Hazards and Forwarding

8.1 Data Hazards

A data hazard occurs when an instruction requires a value produced by an earlier instruction that has not yet been written to the register file. Because several instructions are active at once, the consumer may otherwise receive an old value.

8.2 Forwarding

Forwarding supplies a newer result directly from a later pipeline stage. The forwarding unit compares source register indices in the Execute stage with destination register indices in later stages. The forwarding multiplexer selects either the original operand or a forwarded value.

8.3 Forwarding Sources

The processor uses forwarding paths from EX/MEM and MEM/WB. The EX/MEM forwarding value is selected according to the producer instruction: JAL forwards $PC + 4$, a load-related path selects memory data when available, and the normal arithmetic path selects the ALU result. MEM/WB forwarding uses the final writeback data.

8.4 Load-Use Hazard

A load-use hazard occurs when the instruction immediately following a load requires the loaded value. The value is not available early enough for the dependent instruction's normal execute timing. The hazard detection unit identifies the dependency by comparing the load destination register with the source registers of the instruction in decode.

8.5 Stalling and Bubble Insertion

When a load-use dependency is detected, the processor holds the relevant fetch/decode progress and flushes ID/EX control state to insert a bubble. The load continues through the later stages. Once the loaded value is available, the dependent instruction can proceed.

The top-level stall qualification is:

$$stall = hazard_stall \wedge \neg redirect$$

This gives a redirect priority over a simultaneous hazard stall.

Chapter 9

Branch, Jump and Pipeline Flushing

9.1 Conditional Branches

The implemented conditional branches are BEQ and BNE. The branch comparator evaluates the source operands. If the condition is true, the branch target replaces the sequential PC value.

9.2 JAL Instruction

JAL forms a PC-relative target and writes the sequential return address $PC + 4$ to its destination register. The link value is included in the forwarding data selection so that a later dependent instruction can receive the correct result.

9.3 Redirect Logic

The top-level redirect condition is asserted for a jump or a taken branch. The jump target is selected when the jump control is active; otherwise the branch target is used.

9.4 Pipeline Flushing

A branch or jump may be resolved after sequential-path instructions have entered the pipeline. Those instructions must not execute as valid operations. The fetch stage is flushed on redirect, and ID/EX is flushed on either redirect or stall. Older instructions already in later pipeline stages are allowed to complete.

9.5 Control-Hazard Verification

The branch/jump test places register-writing instructions on paths expected to be skipped. If flushing is correct, the destination registers associated with those wrong-path instructions remain unchanged.

Chapter 10

RTL Module Organization and Integration

10.1 RTL Organization

The RTL source is stored in the `rtl/` directory. The top-level module `risc_v_processor.v` connects the modules into the complete processor.

Table 10.1: RTL module groups

Functional group	Representative modules
Arithmetic	alu.v, alu_operand_mux.v
Decode and control	control_unit.v, decode_stage.v, immediate_generator.v
Fetch	program_counter.v, instruction_memory.v, fetch_stage.v, next_pc.v
Pipeline state	if_id_register.v, id_ex_register.v, ex_mem_register.v, mem_wb_register.v
Hazards	forwarding_unit.v, forwarding_mux.v, hazard_detection_unit.v
Execution	execute_stage.v, branch_comparator.v
Memory and WB	memory_stage.v, data_memory.v, writeback_mux.v
Top-level	register_writeback.v, risc_v_processor.v

10.2 Top-Level Integration

The top-level module connects the fetch, decode, execute, memory, and writeback paths. It also connects the register-file feedback path, forwarding data, hazard detection, and redirect/flush control.

10.3 Project Directory Structure

The project is organized into the following main directories:

```
Pipelined_RISC_V_Processor/
|-- archive/
|-- build/
|-- docs/
|-- rtl/
|-- testbench/
|-- waveforms/
```

The RTL directory contains the hardware modules. The testbench directory contains module-level and integration testbenches. Build stores compiled simulation executables, while waveforms

stores VCD output and the GTKWave configuration.

Chapter 11

Verification and Testing

11.1 Verification Strategy

Verification was performed using individual module testbenches and processor-level integration testbenches. Module-level tests isolate component behavior, while integration tests evaluate interactions across pipeline stages.

11.2 Module-Level Testing

Individual testbenches were created for the ALU, register file, immediate generator, control unit, memories, pipeline registers, forwarding logic, hazard detection, and stage-level modules. These tests help identify local design errors before full integration.

11.3 Processor-Level Testbenches

Three processor-level integration scenarios were used as the main recorded end-to-end checks:

1. Main processor arithmetic and memory program.
2. Branch and jump behavior with wrong-path instructions.
3. Load-use hazard followed by dependent arithmetic.

11.4 Simulation Environment

Icarus Verilog is used to compile the Verilog RTL and testbench. The VVP simulator executes the compiled simulation. Testbenches generate VCD files that can be inspected using GTKWave.

```
iverilog -g2012 -o build/processor_test \
  rtl/*.v testbench/risc_v_processor_tb.v
vvp build/processor_test
```

Chapter 12

Main Processor Integration Results

12.1 Test Program

The default instruction memory program performs the following operations:

1. ADDI loads decimal 10 into $x1$.
2. ADDI loads decimal 20 into $x2$.
3. ADD adds $x1$ and $x2$, placing 30 in $x3$.
4. SW stores $x3$ to data-memory address zero.
5. LW loads the word at address zero into $x4$.

12.2 Observed Results

The simulation produced the following register values:

Table 12.1: Main processor test results

Register	Observed value	Expected value
$x1$	10	10
$x2$	20	20
$x3$	30	30
$x4$	30	30

```
RISC-V PROCESSOR VERIFICATION
Register x1 = 10
Register x2 = 20
Register x3 = 30
Register x4 = 30
PROCESSOR TEST: PASSED
```

12.3 Result Interpretation

The observed values match the expected results. The test exercises immediate arithmetic, register-register addition, register writeback, data-memory write, data-memory read, and load writeback. The testbench reported PASS.

Chapter 13

Branch, Jump and Load-Use Test Results

13.1 Branch and Jump Test

The branch/jump integration program checks a taken BEQ, a not-taken BNE, and a JAL jump. It also includes register-writing instructions on paths expected to be skipped.

Table 13.1: Branch and jump integration results

Register	Observed	Expected
$x1$	5	5
$x2$	5	5
$x3$	0	0
$x4$	7	7
$x5$	28	28
$x6$	0	0
$x7$	11	11

The values in $x3$ and $x6$ remain zero because the instructions intended to be skipped do not update those registers. The value $x5 = 28$ matches the expected JAL link address. The testbench reported PASS.

13.2 Load-Use Hazard Test

The load-use test initializes data memory location zero with 42. The program loads this value into $x1$, calculates $x2 = x1 + 1$, and then calculates $x3 = x1 + x2$.

$$x1 = 42, \qquad x2 = 42 + 1 = 43, \qquad x3 = 42 + 43 = 85$$

Table 13.2: Load-use integration results

Register	Observed	Expected
$x1$	42	42
$x2$	43	43
$x3$	85	85

The final run reported PASS. An earlier run exposed an incorrect dependent result. The register-file implementation was updated to include a same-cycle write/read bypass for matching

nonzero register addresses. After the correction, the test was rerun and the expected values were observed.

Chapter 14

Waveform Analysis, Limitations and Conclusion

14.1 Waveform Analysis

The testbenches generate Value Change Dump files. The main processor waveform is stored as `waveforms/risc_v_processor.vcd`, and the saved GTKWave configuration is `waveforms/risc_v_processor.g`

The waveform can be opened using:

```
gtkwave waveforms/risc_v_processor.vcd
```

Signals useful for inspection include the clock, reset, program counter, instruction, pipeline register contents, stall and flush controls, forwarding selections, ALU inputs and result, memory interface, and writeback signals.

14.2 Limitations

The current implementation has the following limitations:

1. It implements an educational instruction subset rather than the complete RV32I ISA.
2. The recorded integration tests exercise aligned word memory operations; byte and halfword behavior is not established by these results.
3. Exceptions, interrupts, privileged modes, caches, and memory protection are outside the demonstrated scope.
4. Verification is based on directed tests and is not exhaustive formal verification.
5. No synthesis, place-and-route, clock-frequency, power, or silicon-area results are claimed.

14.3 Future Improvements

Future work may include additional RV32I instructions, expanded memory access sizes, more dependency and branch tests, randomized instruction testing, assertions, automated regression execution, and synthesis or FPGA evaluation.

14.4 Conclusion

This project demonstrates the design, implementation, integration, and simulation of an educational 32-bit five-stage pipelined RISC-V processor using Verilog HDL. The design is organized into modular datapath and control components and uses IF/ID, ID/EX, EX/MEM, and MEM/WB pipeline registers.

Forwarding, load-use hazard detection, stalling, branch and jump redirection, and flushing are included to support the instruction sequences verified in simulation. The main processor, branch/jump, and load-use integration tests all passed with their expected register values.

The project provided practical experience in processor architecture, RTL development, pipeline control, simulation, debugging, and waveform inspection. It establishes a foundation for further work in processor design and verification.

Bibliography

[1] RISC-V International, *The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*.

[2] RISC-V International, `https://riscv.org/`.

[3] Icarus Verilog Documentation, `https://steveicarus.github.io/iverilog/`.

[4] GTKWave Documentation, `https://gtkwave.sourceforge.net/`.

[5] Git Documentation, `https://git-scm.com/doc`.

END OF REPORT

5-Stage Pipelined RISC-V Processor